

Kpanda/Gateway API 接口设计

产品信息：

相关 Issue：

- <https://gitlab.daocloud.cn/ndx/engineering/kpanda/-/issues/5230>
- <https://gitlab.daocloud.cn/ndx/engineering/kpanda/-/issues/5253>
- <https://gitlab.daocloud.cn/ndx/engineering/kpanda/-/issues/5265>

原型设计（墨刀）：https://modao.cc/proto/ppaPBiDt951lg559B4n7/sharing?view_mode=read_only

高保证设计（Sketch）：<https://www.sketch.com/s/d9e18f58-c54d-480f-9e7e-97003dcc055a/p/88C2069D-E9B0-4EEC-9ACD-48FD792AB24C/canvas>

Kpanda 默认并不会安装 Gateway API 的相关 CRD，会出现如下的错误。需要手动安装。

根据官方的[文档](#)，使用如下命令安装

代码块

```
1 kubectl apply --server-side -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.1/standard-install.yaml
```

Gateway CRD 安装引导

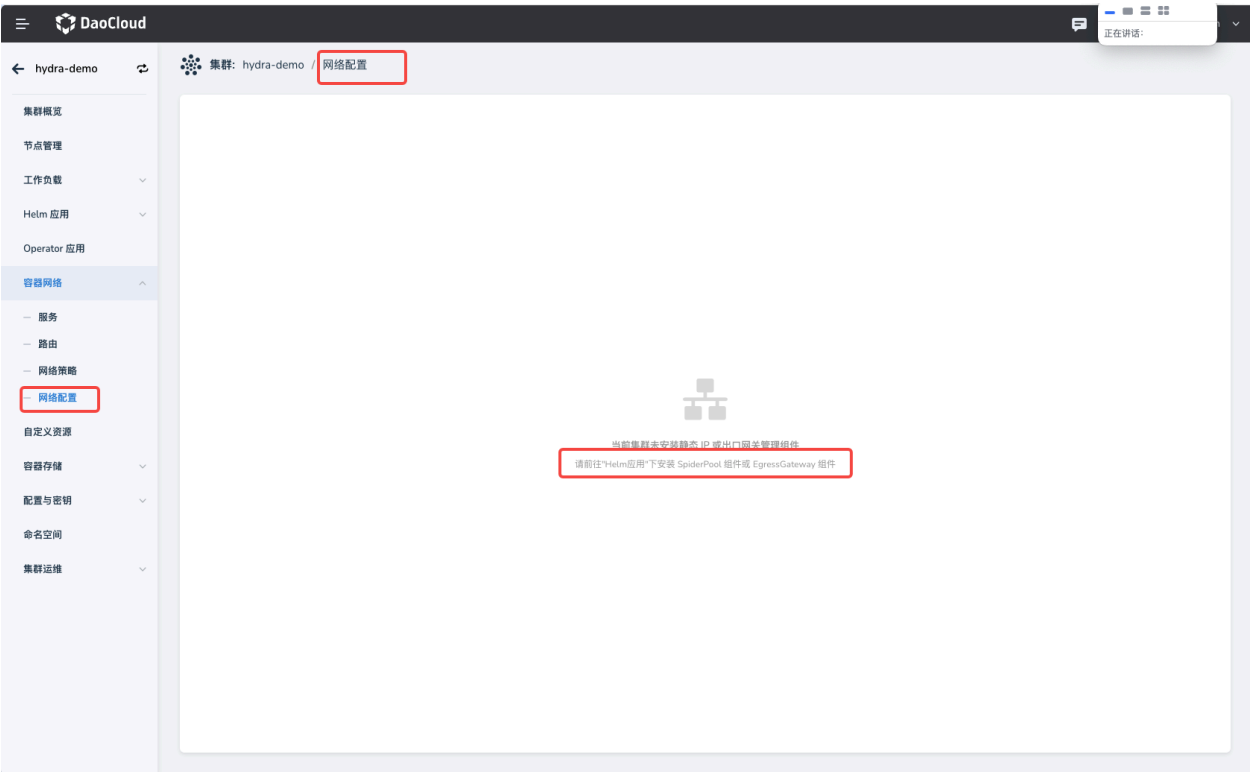
可以参考 "网络配置" 的部分：

会出现如下的错误



张晓珣 4月8日 11:03





实现逻辑：

- Kpanda 在 **get cluster 详情信息**的接口中的 `status.settings.network` 中增加 GatewayAPI 启用的信息。 `{name: "GatewayAPI", enabled: false, intelligentDetection: true, externalAddress: "", setting: "", ...}`
- 然后引导用户跳转到 helm 的页面去安装 GatewayAPI 的 CRDs

通过 <https://10.6.216.82/apis/kpanda.io/v1alpha1/clusters/kpanda-global-cluster> 接口

目前 PR 已经为 addon-pack 准备好了 gateway-api, kgateway 和 higress 的 charts。可以通过 addon 导入。

接口说明（部分）

List Gateways

名称	别名	状态	GatewayClass	External Address	监听端口	命名空间	创建时间	
test-gw	-	● 正常	envoy-gateway	10.23.44.12	80 +1	bs-system	2026-01-18 23:32	⋮

更新

列表中的字段：

1. 别名：展示 `metadata.annotations['kapnda.io/alias-name']` 这个 anno 的 value
2. 状态：enum 类型

External Address: spec.addresses 是数…



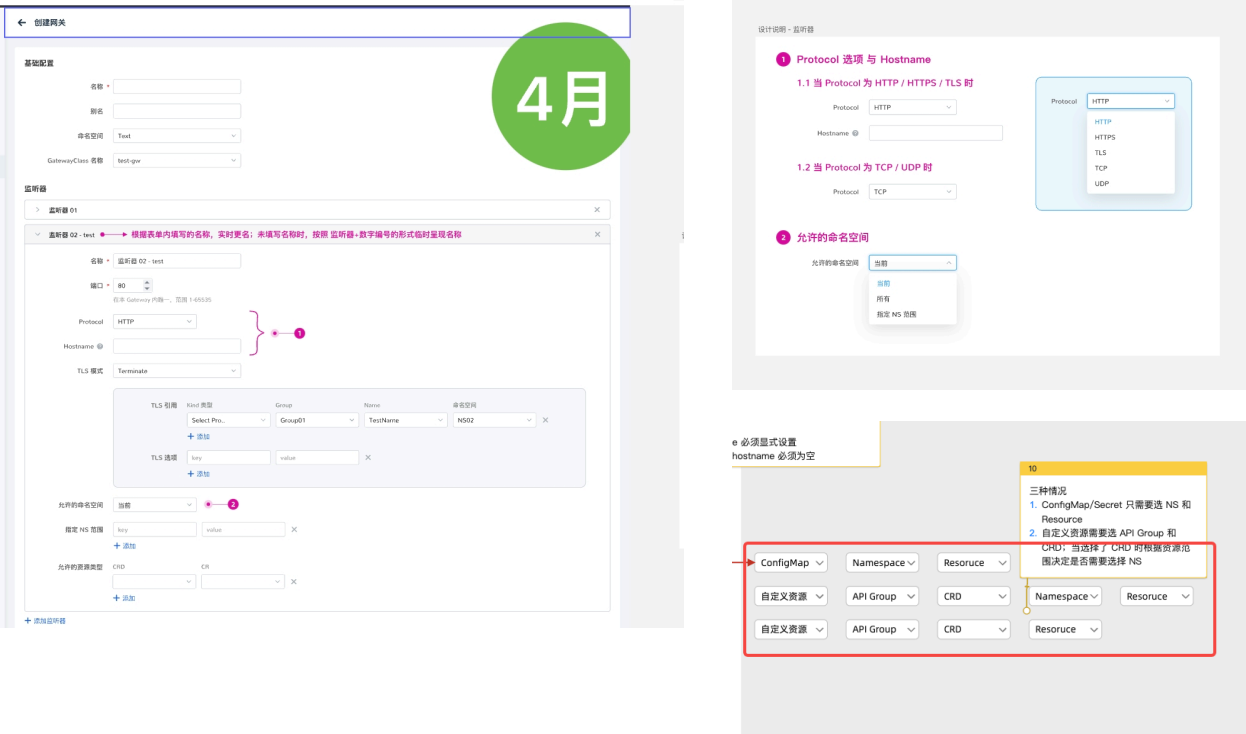
张晓旭 3月6日 17:41

我先按照原型 光展示 VALUE 的数组，等晓妍姐和船长最终设计稿出来再改。

监听端口先按照原型只展示 port

3. GatewayClass: spec.gatewayClassName
4. External Address: spec.addresses 是数组，建议展示 type 和 value
5. 监听端口: spec.listeners 是数组，建议展示 name 和 port

Create Gateway



基础配置

- GatewayClass 名称: spec.gatewayClassName

监听器

监听器字段是 listeners 类型是 Listener 的数组。如果提交空数组，后端会增加一条数据作为默认值。

监听器的字段：

- 名称: spec.listeners[].name
- 端口: spec.listeners[].port 取值范围是 （1~65535）
- 协议: spec.listeners[].protocol 协议里的 enum 类型 ProtocolType，支持 HTTP，HTTPS，TLS，TCP，UDP
- Hostname: spec.listeners[].hostname 域名风格的字符串；仅在 HTTP，HTTPS，TLS 有效。
- TLS 模式: 是复杂的 ListenerTLSConfig 类型；
 - TLS 模式: spec.listeners[].tls.mode ；值是 enum 类型：Terminate, Passthrough
 - TLS 引用: spec.listeners[].tls.certificateRefs ；值是一个数组，数组的元素是 Kubernetes 的资源对象 SecretObjectReference 类型；

ConfigMap

spec.listeners[].tls.certificateRefs[].kind 为 ConfigMap

代码块

```
1 spec:
2   listeners:
3     - name: https
4       protocol: HTTPS
5       port: 443
```

		<pre>6 hostname: 7 "example.com" 8 tls: 9 mode: Terminate # 终 10 止 TLS, 这是最常用的模式 11 certificateRefs: 12 - name: example- 13 com-tls-configmap # 引用包 14 含证书的 K8s Secret 15 kind: ConfigMap 16 group: "" # 留空表 17 示核心 API 组 18 # namespace: 19 Optional (默认为 Gateway 20 所在的命名空间) 21 namespace: 22 namespace</pre>
Secret	spec.listeners[].tls. certificateRefs[].kind 为 Secret	代码块 <pre>1 spec: 2 listeners: 3 - name: https 4 protocol: HTTPS 5 port: 443 6 hostname: 7 "example.com" 8 tls: 9 mode: Terminate # 终 10 止 TLS, 这是最常用的模式 11 certificateRefs: 12 - name: example- 13 com-tls-secret # 引用包含证 14 书的 K8s Secret 15 kind: Secret 16 namespace: 17 namespace 18 group: "" # 留空表 19 示核心 API 组 20 # namespace: 21 Optional (默认为 Gateway 22 所在的命名空间)</pre>
自定义资源	spec.listeners[].tls. certificateRefs[].group up - API 组: group - 资源类型: kind - 命名空间: namespace - 资源名称: name	代码块 <pre>1 spec: 2 gatewayClassName: 3 enterprise-gateway-class 4 listeners: 5 - name: https 6 protocol: HTTPS 7 port: 443 8 hostname: 9 "api.example.com" 10 tls: 11 mode: Terminate 12 certificateRefs: 13 - group: cert- 14 manager.io # 自定义资 15 源的 API 组</pre>

		12 kind: Certificate # 自 定义资源的类型 13 name: my-website- cert # 具体的资源名称 14 # namespace: Optional (默认为 Gateway 所在的命名空间)
--	--	---

- TLS 选项: `spec.listeners[].tls.options` ; 值是 Object 类型, key 为 Annotation 的 key 的规则; value 为 Annotation 的 value 规则;

• 允许的命名空间:

`spec.listeners[].allowedRoutes.namespaces.from` : 值是

`from` : enum 类型; 有 3 个值分别是 `All` , `Selector` , `Same`

- `selector` : 只有当 `from` 字段为 `Selector` 的时候才生效。

UI 字段	HTTPRoute Spec	说明
当前	<code>namespaces.from</code> 为 <code>Same</code>	代码块 1 allowedRoutes: 2 namespaces: 3 from: Same # 仅限同命名空间
所有	<code>namespaces.from</code> 为 <code>All</code>	代码块 1 allowedRoutes: 2 namespaces: 3 from: All # 允 许所有命名空间
指定 命名空间 范围	<code>namespaces.from</code> 为 <code>Selector</code> 且 <code>namespace.selector.matchLabels</code> 这个 Object 的 key 和 value 满足	代码块 1 allowedRoutes: 2 namespaces: 3 from: Selector 4 selector: 5 matchLabels: 6 exposure: public # 只有打了这个 标签的 Namespace 下的路由才能连接

- form 的值是 enum 类型, 分别是Enum: [All Selector Same] (注意大小写)
- `Same` : 仅限同命名空间
- `All` : 允许所有命名空间
- `Selector` 制定 NS 范围:

- 允许的资源类型: `spec.listeners[].allowedRoutes.kinds` : 值是 `RouteGroupKind` 类型.

配置参数: spec.parametersRef 是一个...



卢传佳 3月5日 17:36

这个我的想法是走 Kind/Name , 如果是跨 NS, 可以用 Kind/Name (NS) , 而且后期还可以加上资源跳转



张晓珣 3月5日 17:39

List GatewayClasses

Q 搜索 YAML 创建 创建					
名称	别名	状态	控制器	配置参数	创建时间
envoy-internet	-	Accepted	gateway.envoyproxy.io/g ayclass-controller		2026-01-18 12:32

列表中的字段：

1. 别名：展示 `metadata.annotations['kapnda.io/alias-name']` 这个 anno 的 value
2. 状态：enum 类型
3. 控制器： `spec.controllerName` 字符串，原样展示
4. 配置参数： `spec.parametersRef` 是一个资源的名字（可能是 ConfigMap，也可能是自定义资源）。

a. 根据产品建议，以 ConfigMap 为例，展示为 `ConfigMap/default-gc-config(gateway-system)`

Create GatewayClass

//

@卢传佳 ok，那以 configmap 为例，就是 `ConfigMap/default-gc-config(gateway-system)` ？

卢传佳 4月23日 13:50

yes

应该不是下拉框，而是输入框吧？

卢传佳 4月23日 13:49

这个最好是让用户选择，controllerName 让用户输入比较苦难的，而且这个应该是可以查出来的

//

张晓珣 4月23日 13:56

这个基于什么来查询？不同的 gateway 方案的值 是各不相同的。

卢传佳 4月23日 13:57

我来回忆下

名称 *

别名

Controller 名称 *

2

张晓旭 拉选择集群已有，创建后不可修改

空间 up

引用资源

☒ ConfigMap

☐ Secret

☐ 自定义资源

命名空间

test-gw

资源名称

test-gw

引用资源

☐ ConfigMap

☐ Secret

☒ 自定义资源

API 组

test-gw

资源类型

test-gw

命名空间

test-gw

资源名称

test-gw

9

仅当资源类型为 Namespace 范围时，才需要选择命名空间

列表中的字段：

1. Controller 名称： `spec.controllerName` 是一个 域名风格 的字符串，用于标识实现该 GatewayClass 的控制器，格式为 `<domain>/<path>`，例如 `gateway.envoyproxy.io/gatewayclass-controller`、`istio.io/gateway-controller`、`k8s.io/ingress-nginx`。一旦创建后不可变更。

a. 应该不是下拉框，而是输入框吧？ 卢传佳
2. 引用资源： `spec.parametersRef` 是一个资源信息

UI 字段	HTTPRoute Spec	说明
ConfigMap	<code>spec.parametersRef</code>	代码块 <pre>1 spec: 2 parametersRef: 3 group: "" # 核心 API 组 (ConfigMap 属于此组) 4 kind: ConfigMap 5 name: nginx-global-config 6 namespace: gateway- system # 必须指定命名空间</pre>
Secret	<code>spec.parametersRef</code>	代码块 <pre>1 spec:</pre>

所属网格：spec.parentRefs 是数组，是…

张晓旭 3月6日 18:40

这个是所属网关？ 还是网格？

// 张晓珣 3月6日 18:40

		<pre>2 parametersRef: 3 group: "" # 核心 API 组 4 kind: Secret 5 name: cloud-provider- credentials 6 namespace: kube-system</pre>
自定义资源	spec.parametersRef - API 组: group - 资源类型: kind - 命名空间: namespace - 资源名称: name	代码块 <pre>1 spec: 2 parametersRef: 3 group: gateway.envoyproxy.io # 厂商定 义的 API 组; 注意不用标记版本 4 kind: EnvoyProxy # 厂商定义的 CRD 类型 5 name: custom-proxy-config 6 namespace: envoy-gateway- system</pre>

自定义资源通过 CRDs 的接口获取：

1. API 组下拉框（group）：通过 Apiextensions 的 ListCustomResourceDefinitionGroups 接口获取 Group 信息
2. 资源类型下拉框（Kind）：通过 Apiextensions 的 ListCustomResourceDefinitions 接口 CRD 的信息；其中 group 来自于上一层接口处理
- a. 一个 CRD 中可能存在多的版本，在返回值的 .versions 数组中，过滤 `v.deperated == false && v.served == true && v.storage == true` 的版本。
- b. 返回数组的元素中，通过 `metadata.scope` 的枚举值： `NAMESPACED` 和 `CLUSTER` 来判断是否是 namespace 级别的资源

List Routes

Gateway							
Route							
GatewayClass							
Tab1-4							
Tab1-5							
Q 搜索 YAML 创建 创建							
名称	路由类型	状态	所属网关	域名匹配	绑定服务	命名空间	创建时间
http-route	HTTPRoute	● 正常	test-gw	hr.daocloud.test	ht-backend	bs-system	2026-01-18 23:32
http-route	TCPRoute	● 正常	test-gw	hr.daocloud.test	ht-backend(NS)	bs-system	2026-01-18 23:32
http-route	GRPCRoute	● 正常	test-gw	hr.daocloud.test	ht-backend(NS)	bs-system	2026-01-18 23:32

列表中的字段：

1. 路由类型：enum 类型
2. 状态：enum 类型
3. 所属网格： `spec.parentRefs` 是数组，是一个 Kube 资源信息

所属 Gateway

parentRefs: - name: my-gateway # 引用...

张晓旭 3月30日 11:45

这个是个数组，前端展示/填充第几个呢？ @卢传佳

卢传佳 3月30日 11:46

@张晓珣 help

// 张晓珣 3月30日 11:51 （编辑过）

@张晓旭 假设，如果是通过 UI 创建的，理论上数组只有一个元素。但如果绕开 UI 创建的实例，在编辑过程中，仅展示第一个元素，确实不合适。

张晓旭 3月30日 14:25

@卢传佳 怎么说？要不先展示第一个，记个 issue？

卢传佳 3月30日 14:26

可以记一个 issue

张晓旭 3月30日 14:27

@卢传佳 ok

卢传佳 3月30日 14:28

<https://gitlab.daocloud.cn/ndx/engineering/kpanda/-/issues/5291>

[图片]

// 张晓珣 3月30日 10:09

4. **域名匹配:** `sepc.hostnames` 是一个字符串数组;
5. **绑定服务:** `spec.rules` 是一个规则数组, GRPCBackendRef 和 HTTPBackendRef 的服务, 对应的值也是 Kube 资源信息

Create Routes / HTTPRoute

基础信息

基础配置

名称

别名

命名空间

Select

路由类型

HTTPRoute

网关指向

☒ 本命名空间 ☐ 其他命名空间

网关

Select

域名

留空则匹配所有域名

+ 添加

```
1  kind: HTTPRoute
2  metadata:
3    name: foo-route
4    namespace: default
5  spec:
6    parentRefs:
7      - name: my-gateway # 引用
                          所属的 Gateway 名称
8      namespace: bee-route #
                          如果是其他 ns 的, 需要额外说明
```

包含 metadata 和 spec 字段的信息

UI 字段	HTTPRoute Spec	说明
网关	<code>parentRefs[] . name</code>	parentRefs 字段： <code>name</code>：Gateway 的 name <code>namespace</code>：当选择 “「其他命名空间」” 时，需要提供 namespace，否则就是 <code>metadata.namespace</code> 的值。
域名	<code>hostnames</code>	hostnames 字段： 是 字符串的数组。

规则配置 (HTTPRoute)

规则配置

> 规则 01

> 规则 02

流量匹配条件

路径

前缀匹配 Path 匹配项, 如: /user 或 /user/add

请求方法

请求头

Header 关键字

可以由小写字母组成

+ 添加参数

参数匹配

参数名

可以由小写字母组成

+ 添加参数

判断条件

匹配条件

匹配值

判断条件

匹配条件

匹配值

```

1  spec:
2    rules:
3      - matches:
4        # 场景 1: 组合匹配 (AND
      关系)
5        # 只有当路径前缀是 /api,
      方法是 POST, 且包含特定的
      Header 时才匹配
6      - path:
7          type: PathPrefix
8          value: /api
9          method: POST
10         headers:
11       - type: Exact
12         name: "X-API-
      Version"
13         value: "v2"

```

“
@卢传佳 Route 可以设置
`name`，后期可以支持让用户
替换“规则 01”这个名字。
卢传佳 3月30日 11:46
okay

```
14     queryParams:
15     - type: Exact
16       name: "debug"
17       value: "true"
18
19     # 场景 2: 路径正则匹配
20     (部分控制器支持)
21     - path:
22       type:
23         RegularExpression
24       value: "/users/[0-9]+$"
25
26     method: GET
27
28     # 场景 3: 仅匹配特定
29     Header (不限路径)
30     - headers:
31     - type: Exact
32       name: "Environment"
33       value: "canary"
34
35     backendRefs:
36     - name: api-service-v2
37       port: 8080
```

流量匹配条件 (Matching Rules)

对应规范中的 `spec.rules[].matches`

	UI 字段	HTTPRouteMatche Spec	说明
1	路径 (Path)	matches[].path	<code>type</code> :如 <code>PathPrefix</code> , <code>Exact</code> <code>value</code> :路径的字符串
2	请求方法	matches[].method	对应 HTTP 方法，Enum类型: [GET HEAD POST PUT DELETE CONNECT OPTIONS TRACE PATCH]
3	请求头 (Headers)	matches[].headers	包含 <code>name</code> , <code>value</code> 和 <code>type</code> (如 <code>Exact</code> , <code>RegularExpression</code>)。
4	参数匹配 (Query Params)	matches[].queryParams	包含 <code>name</code> , <code>value</code> 和 <code>type</code> (如 <code>Exact</code> , <code>RegularExpression</code>)。

过滤器 (Filters)

对应规范中的 `spec.rules[].filters` ，即 `HTTPRouteFilter` 类型。这些通常用于在请求被转发到后端之前或之后修改请求。

过滤器 编辑

请求头重写 ☒ 启用

动作

▼

 关键字 值 ✕

关键字必须是小写字母，数字字母或“-”组成，并且必须以字母开头及字母或数字字母结尾

+ 添加

响应头重写 ☒ 启用

动作

▼

 关键字 值 ✕

关键字必须是小写字母，数字字母或“-”组成，并且必须以字母开头及字母或数字字母结尾

+ 添加

路径重写 ☒ 启用

原路径

示例: /blog1

重写路径

示例: /blog1

代码块

```
1 spec:
2   rules:
3     - matches:
4       - path: { type: PathPrefix, value: "/old-api" }
```

```
5      filters:
6        # --- 1. 修改请求头
        (RequestHeaderModifier) ---
7        - type:
RequestHeaderModifier
8
      requestHeaderModifier:
9        add:
10         - name: "X-
Internal-Source"
11         value:
"gateway-api"
12        set:
13         - name: "X-App-
Id"
14         value:
"shopping-cart-v1"
15        remove: ["X-Debug-
Token"]
16
17        # --- 2. 路径重写
        (URLRewrite) ---
18        # 将 /old-api/users 重写
        为 /v2/users 发给后端
19        - type: URLRewrite
urlRewrite:
20        path:
21        type:
ReplacePrefixMatch
22        replacePrefix:
23        "/v2"
24
25        # --- 3. 响应头修改
        (ResponseHeaderModifier) --
        -
26        - type:
ResponseHeaderModifier
27
      responseHeaderModifier:
28        add:
29         - name: "Cache-
Control"
30         value: "no-
cache"
31
```

看原型，仅支持 ReplacePrefixMatch 类…



张晓珣 4月23日 11:54

只有 ReplaceFullPath
ReplacePrefixMatch 这 2 种 会
支持 路径重写 (URLRewrite)

原生字段不包含 "启用" 的部分。`spec.rules[].filters` 的值是数字，根据 数组的 type 来判断是否 “启用”。

UI 字段	HTTPRouteFilter Spec	说明
请求头重写	<code>type: RequestHeaderModifier</code>	requestHeaderModifier 字段： 动作： 包含 <code>set</code> ， <code>add</code> ， <code>remote</code> 3 种。其中： 1. <code>set</code> 和 <code>add</code> 是数组，包含 <code>name</code> 和 <code>value</code> 的字段。 2. <code>remote</code> 是字符串数组
	<code>type: URLRewrite</code>	urlRewrite 字段：

响应头重写		<code>path.type</code>：字段是 Enum 包含 <code>ReplaceFullPath</code> 和 <code>ReplacePrefixMatch</code> 2 种 <code>path.replacePrefix</code>：字段是替换前缀看原型，仅支持 <code>ReplacePrefixMatch</code> 类型。
路径重写	<code>type:</code> <code>ResponseHeaderModifier</code>	responseHeaderModifier 字段： 同 requestHeaderModifier 字段

后端服务 (HTTPRoute Backend Refs)

后端服务

目标服务

服务名称

contour-new-crd-gtw

选择服务

权重

100

%

流量镜像

未启用

流量镜像选取的服务将不再参与负载均衡，但会收到全部流量请求

+ 添加

默认必填，所以第一个目标服务不带删除按钮，从添加第二个目标服务开始带删除

对应规范中的 `spec.rules[].backendRefs`

UI 字段	HTTPRoute Spec	说明
服务名称	<code>backendRefs[].name</code>	目标 Service 的名称： <code>name</code>：Service 的 name； <code>weight</code>：服务的权重
权重	<code>backendRefs[].weight</code>	即上述的 权重
流量镜像	<code>filters[type="RequestMirror"]</code>	看描述

因为「流量镜像」的能力是通过，filters 实现的，下面为示例：

流量镜像

```
1 spec:
2   parentRefs:
3   - name: my-gateway
4   rules:
5   - matches:
6     - path:
7       type: PathPrefix
8       value: /api
9   filters:
10  - type: RequestMirror
11    requestMirror:
12      backendRef:
13        name: mirror-service # 镜像接收者
14        port: 8080
15  backendRefs:
16  - name: sandbox-service # 实际业务处理者
17    port: 80
```

和 HTTPRouteMatch 不同，GPRCRouteMatch 的 type 是 enum 类型只有 2 个值：Exact 和 ...



张晓旭 3月30日 14:52 （编辑过）

所以，如果用户选择 GPRC 的话，不需要填写 路径和参数匹配

type: enum 类型只有 2 个值：Exact 和 ...



张晓旭 3月30日 18:12

应该是这个结构



所以，开启「流量镜像」的能力时，需要额外设置一条 type 类型为 **RequestMirror** 的 filters。

Create Routes / GRPCRoute

规则配置（GRPCRoute）

流量匹配条件 (Matching Rules)

代码块

```
1 spec:
2   rules:
3     - matches:
4       - method: # 只有一个值，不是数字，相当于 path 的用法
5         type: Exact
6         service: "com.example.ChatService" # 完整的 gRPC 服务名
7         method: "SendMessage"             # 方法名
8         headers:                           # gRPC 使用 HTTP/2
9           # Header 传递元数据
10          - name: "x-user-role"
11            value: "premium"
12            type: Exact
```

和 HTTPRouteMatch 不同，GRPCRouteMatch 只有 headers 和 method（且 method 和 HTTP method 是同名不同实质），没有 path 和 queryParams。

可能的 UI	GRPCRouteMatch Spec	说明
方法	<code>matches[].method</code>	<code>type</code>：enum 类型只有 2 个值：Exact 和 RegularExpression <code>name</code>：字符串 <code>value</code>：字符串
请求头 (Headers)	<code>matches[].headers</code>	是 <i>GRPCHeaderMatch</i> 数组，字段： <code>type</code>：enum 类型只有 2 个值：Exact 和 RegularExpression <code>name</code>：字符串 <code>value</code>：字符串

过滤器 (Filters)

和 HTTPRouteFilterType 类型相比，支持：ResponseHeaderModifier，RequestHeaderModifier 和 RequestMirror（无 `URLRewrite`），字段用法和 HTTP 的部分一模一样。

后端服务 (Backend Refs)

应规范中的 `spec.rules[].backendRefs`

UI 字段	HTTPRoute Spec	说明
服务名称	<code>backendRefs[].name</code>	目标 Service 的名称：

RequestMirror（无 URLRewrite）



张晓旭 3月30日 14:55

设计图中是不是没有 requestMirror?



张晓旭 3月30日 14:58

@张晓旭 这个跟后端服务里，流量镜像 requestMirror 是一个东西吗？

URLRewrite



张晓旭 3月30日 14:55

GPCRC 不需要显示路径重写

		<code>name</code>：Service 的 name； <code>weight</code>：服务的权重
权重	<code>backendRefs[].weight</code>	即上述的 权重

权限说明 🔥

1. ns-admin 仅可以查询到自己有权限的 ns 下的 gateway
2. ns-admin 创建 gateway 时，前端可以
 - a. 调用 `ListAccessibleGatewayClasses` 查询到自己 ns 所在的 cluster 的 gatewayclass
 - b. 调用 `ListGatewayClasses` 接口仅对 cluster 用户生效

CRDs 示例

GatewayClass 示例

GatewayClass 示例

```
1  apiVersion: gateway.networking.k8s.io/v1
2  kind: GatewayClass
3  metadata:
4    name: cluster-gateway
5  spec:
6    controllerName: "example.net/gateway-controller"
```

Gateway CR 示例

Gateway CR 示例

```
1  apiVersion: gateway.networking.k8s.io/v1
2  kind: Gateway
3  metadata:
4    name: kgateway-sample
5    namespace: kgateway-system
6  spec:
7    # 指定使用的 GatewayClass，通常由云厂商或控制面（如 Istio, Cilium,
8    gatewayClassName: kgateway
9
10   listeners:
11     # 1. 标准 HTTP 监听器（通常用于重定向到 HTTPS）
12     - name: http
13       port: 80
14       protocol: HTTP
15       allowedRoutes:
16         namespaces:
17           from: Same # 仅允许当前命名空间的路由（例如用于全站强制跳转配
18           置）
19
20     # 2. 生产环境 API 业务（HTTPS）
21     - name: https-api
22       port: 443
```

```
22     protocol: HTTPS
23     hostname: "api.example.com"
24     tls:
25       mode: Terminate # 在网关层终止 TLS
26       certificateRefs:
27         - name: api-example-com-tls
28     allowedRoutes:
29       namespaces:
30         from: Selector
31         selector:
32           matchLabels:
33             exposed-to-gateway: "true" # 允许带有此标签的命名空间挂载
路由
34
35     # 3. 泛域名支持 (用于多租户或测试环境)
36     - name: https-wildcard
37       port: 443
38       protocol: HTTPS
39       hostname: "*.dev.example.com"
40       tls:
41         mode: Terminate
42         certificateRefs:
43           - name: wildcard-dev-tls
44       allowedRoutes:
45         namespaces:
46           from: All # 允许集群内所有命名空间关联
47
48     # 4. 专门的 gRPC 流量监听器
49     - name: grpc-internal
50       port: 50051
51       protocol: HTTPS # gRPC 常用 TLS
52       hostname: "grpc.internal.example.com"
53       tls:
54         mode: Terminate
55         certificateRefs:
56           - name: grpc-tls
57       allowedRoutes:
58         namespaces:
59           from: Same
60
61     # 基础架构层面的地址配置 (可选, 取决于控制器支持情况)
62     addresses:
63       - type: IPAddress
64         value: 1.2.3.4
```

Gateway HTTP 示例

过程问题

1. 需要确定是否会默认创建一个 rules，如下图


```
> JSON.parse("{\"apiVersion\":\"gateway.networking.k8s.io/v1\",\"kind\":\"HTTPRoute\",\"metadata\":{\"name\":\"sdasd\",\"namespace\":\"default\"},\"spec\":{\"parentRefs\":[{\"group\":\"gateway.networking.k8s.io\",\"kind\":\"Gateway\",\"name\":\"example-vivian\"}]}}")
{
  apiVersion: 'gateway.networking.k8s.io/v1', kind: 'HTTPRoute', metadata: {name: 'sdasd', namespace: 'default'}, spec: {parentRefs: [
    {group: 'gateway.networking.k8s.io', kind: 'Gateway', name: 'example-vivian'}
  ]}
}

{
  apiVersion: 'gateway.networking.k8s.io/v1', kind: 'HTTPRoute', metadata: {name: 'sdasd', namespace: 'default'}, spec: {parentRefs: [
    {group: 'gateway.networking.k8s.io', kind: 'Gateway', name: 'example-vivian'}
  ]}
}

{
  apiVersion: 'gateway.networking.k8s.io/v1', kind: 'HTTPRoute', metadata: {creationTimestamp: '2026-04-01T06:26:41Z', generation: 1, name: 'sdasd', namespace: 'default', resourceVersion: '259111543', uid: '4df37670-bc1b-4b65-bea0-412c3c74a5a7'}, spec: {parentRefs: [
    {group: 'gateway.networking.k8s.io', kind: 'Gateway', name: 'example-vivian'}
  ], rules: [
    {matches: [
      {path: {type: 'PathPrefix', value: '/'}}
    ]}
  ]}
}
```

我 POST 的

返回的

答复：创建 HTTPRoute 的时候，如果没有显示给 rules 的时候，会自动创建一个 default 的 rule：

代码块

```
1 rules:
2   - matches:
3     - path:
4       type: PathPrefix
5       value: /
```